Q1. Employee Salary Analysis A company records the monthly salaries of its employees, in thousands of rupees.

```
salaries = [    18, 20, 22, 22, 24,    25, 25, 26, 28, 30,    32, 35, 38, 42, 150]
```

```
# (a) Write Python functions to calculate the mean, median, and all modes without using statistics.mean(), statistics.median
# mean
def calculated_mean(data):
    return sum(data) / len(data)

def calculated_median(data):
    sorted_data = sorted(data)
    n = len(sorted_data)

    if n % 2 != 0:
        return sorted_data[n // 2]
    else:
        mid1 = sorted_data[n // 2 - 1]
        mid2 = sorted_data[n // 2]
        return (mid1 + mid2) / 2

def calculated_mode(data):
    frequency = {}

    for i in data:
        frequency[i] = frequency.get(i, 0) + 1

    max_frequency = max(frequency.values())

    if max_frequency == 1:
        return []

    mode = []
    for key, value in frequency.items():
        if value == max_frequency:
            mode.append(key)

    return mode


print("Mean:", calculated_mean(salaries))
print("Median:", calculated_median(salaries))
print("Mode:", calculated_mode(salaries))
```

```
Mean: 35.8
Median: 26
Mode: [22, 25]
```

```
# b) Remove the salary of 150 and recalculate all three measures. Calculate the percentage change in
# the mean and median, then justify which measure better represents a typical employee's salary.
salaries = [18, 20, 22, 22, 24,
            25, 25, 26, 28, 30,
            32, 35, 38, 42, 150]

old_mean = calculated_mean(salaries)
old_median = calculated_median(salaries)


new_salaries = salaries.copy() # Remove salary 150
new_salaries.remove(150)


new_mean = calculated_mean(new_salaries)  #Recalculate measures
new_median = calculated_median(new_salaries)
new_mode = calculated_mode(new_salaries)

# Percentage change
mean_change = abs((new_mean - old_mean) / old_mean) * 100
median_change = abs((new_median - old_median) / old_median) * 100

print("New Mean:", new_mean)
print("New Median:", new_median)
print("New Mode:", new_mode)

print("Mean Percentage Change:", mean_change, "%")
```

```
print("Median Percentage Change:", median_change, "%")
```

```
New Mean: 27.642857142857142
New Median: 25.5
New Mode: [22, 25]
Mean Percentage Change: 22.785315243415795 %
Median Percentage Change: 1.9230769230769231 %
```

Q2. Customer Spending Analysis Using the Tips Dataset Use Seaborn's built-in tips dataset.

```
import seaborn as sns
import pandas as pd
```

```
df=sns.load_dataset('tips')
```

```
# (a) Calculate the mean, median, and all modes of total_bill and tip. Then divide the
# dataset into Lunch and Dinner groups and calculate the mean and median bill for each group.

def calculated_mean(data):
    return sum(data) / len(data)

def calculated_median(data):
    sorted_data = sorted(data)
    n = len(sorted_data)

    if n % 2 != 0:
        return sorted_data[n // 2]
    else:
        mid1 = sorted_data[n // 2 - 1]
        mid2 = sorted_data[n // 2]
        return (mid1 + mid2) / 2

def calculated_mode(data):
    frequency = {}

    for i in data:
        frequency[i] = frequency.get(i, 0) + 1

    max_frequency = max(frequency.values())

    if max_frequency == 1:
        return []

    mode = []
    for key, value in frequency.items():
        if value == max_frequency:
            mode.append(key)

    return mode

# Mean, median and mode of total_bill
print("TOTAL BILL")
print("Mean:", calculated_mean(df["total_bill"]))
print("Median:", calculated_median(df["total_bill"]))
print("Modes:", calculated_mode(df["total_bill"]))

# Mean, median and mode of tip
print("\nTIP")
print("Mean:", calculated_mean(df["tip"]))
print("Median:", calculated_median(df["tip"]))
print("Modes:", calculated_mode(df["tip"]))

# Lunch and Dinner groups
for meal in ["Lunch", "Dinner"]:
    group = df[df["time"] == meal]["total_bill"].tolist()

    print("\n", meal)
    print("Mean Bill:", calculated_mean(group))
    print("Median Bill:", calculated_median(group))
```

```
TOTAL BILL
Mean: 19.785942622950817
Median: 17.795
Modes: [13.42]

TIP
Mean: 2.9982786885245902
```

```
Median: 2.9
Modes: [2.0]

 Lunch
Mean Bill: 17.168676470588235
Median Bill: 15.965

 Dinner
Mean Bill: 20.79715909090909
Median Bill: 18.39
```

```
# (b) Identify the three largest bills, temporarily exclude them, and recalculate the
# overall mean and median. Use Python to compare the results before and after exclusion, and explain which measure better r

original_mean = calculated_mean(df["total_bill"])
original_median = calculated_median(df["total_bill"])

largest_bills = sorted(df["total_bill"], reverse=True)[:3]

print("Three Largest Bills:", largest_bills)

filtered_bills = df[
    ~df["total_bill"].isin(largest_bills)
]["total_bill"].tolist()

new_mean = calculated_mean(filtered_bills)
new_median = calculated_median(filtered_bills)


mean_change = abs(
    (new_mean - original_mean) / original_mean
) * 100

median_change = abs(
    (new_median - original_median) / original_median
) * 100

print("\nBefore Exclusion")
print("Mean:", original_mean)
print("Median:", original_median)

print("\nAfter Exclusion")
print("Mean:", new_mean)
print("Median:", new_median)

print("\nPercentage Change")
print("Mean Change:", mean_change, "%")
print("Median Change:", median_change, "%")
```

```
Three Largest Bills: [50.81, 48.33, 48.27]

Before Exclusion
Mean: 19.785942622950817
Median: 17.795

After Exclusion
Mean: 19.42058091286307
Median: 17.59

Percentage Change
Mean Change: 1.8465721702030182 %
Median Change: 1.1520089912896985 %
```

Q3. Build a Central Tendency Calculator From Scratch Use the following datasets:

```
data_1 = [12, 15, 15, 18, 20, 22, 25]
data_2 = [5, 5, 10, 10, 15, 20]
data_3 = [7, 14, 21, 28, 35]
data_4 = [100, 100, 100, 100]
data_5 = [1, 2, 3, 4, 5, 100]
```

```
# (a) Build reusable Python functions to calculate the mean, median, and mode
# without using built-in statistical functions. Your program must handle unsorted data, odd and even dataset lengths, multi

# 1. Mean
def calculated_mean(data):
    return sum(data) / len(data)


# 2. Median
def calculated_median(data):
```

```
    sorted_data = sorted(data)
    n = len(sorted_data)

    if n % 2 != 0:
        return sorted_data[n // 2]
    else:
        mid1 = sorted_data[n // 2 - 1]
        mid2 = sorted_data[n // 2]
        return (mid1 + mid2) / 2


# 3. Mode
def calculated_mode(data):
    frequency = {}

    for i in data:
        frequency[i] = frequency.get(i, 0) + 1

    max_frequency = max(frequency.values())

    if max_frequency == 1:
        return []

    mode = []

    for key, value in frequency.items():
        if value == max_frequency:
            mode.append(key)

    return mode
datasets = {
    "Data 1": data_1,
    "Data 2": data_2,
    "Data 3": data_3,
    "Data 4": data_4,
    "Data 5": data_5
}


# 4. Formatted report
print(f"{'Dataset':<10} {'Mean':<10} {'Median':<10} {'Mode'}")
print("-" * 45)

for name, data in datasets.items():
    mean = calculated_mean(data)
    median = calculated_median(data)
    mode = calculated_mode(data)

    if mode == []:
        mode = "No mode"

    print(f"{name:<10} {mean:<10.2f} {median:<10.2f} {mode}")
```

```
Dataset    Mean       Median     Mode
---------------------------------------------
Data 1     18.14      18.00      [15]
Data 2     10.83      10.00      [5, 10]
Data 3     21.00      21.00      No mode
Data 4     100.00     100.00     [100]
Data 5     19.17      3.50       No mode
```

```
# (b) Run your calculator on all five datasets and generate a formatted report.
#  Identify which dataset is most affected by an extreme value and demonstrate numerically why its mean and median behave d
original_data = data_5
filtered_data = [x for x in original_data if x != 100]

print("\nData 5: Before and After Removing 100")
print("Original Mean:", calculated_mean(original_data))
print("Original Median:", calculated_median(original_data))

print("New Mean:", calculated_mean(filtered_data))
print("New Median:", calculated_median(filtered_data))
```

```
Data 5: Before and After Removing 100
Original Mean: 19.166666666666668
Original Median: 3.5
New Mean: 3.0
New Median: 3
```

Q4. Statistical Transformations and Reasoning An online store records the number of products purchased per order.

```
# (a) Calculate the mean, median, and all modes. Remove the largest observation and recalculate
# the measures. Use the results to explain how an extreme order affects each measure.

orders = [
    1, 1, 1, 2, 2, 2, 2, 3, 3, 4,
    4, 5, 5, 6, 8, 10, 15, 20, 25, 80
]

def calculated_mean(data):
    return sum(data) / len(data)

def calculated_median(data):
    s = sorted(data)
    n = len(s)

    if n % 2 != 0:
        return s[n // 2]
    else:
        return (s[n // 2 - 1] + s[n // 2]) / 2

def calculated_mode(data):
    frequency = {}

    for x in data:
        frequency[x] = frequency.get(x, 0) + 1

    max_frequency = max(frequency.values())

    if max_frequency == 1:
        return []

    return [x for x, count in frequency.items()
            if count == max_frequency]

print("Original Mean:", calculated_mean(orders))
print("Original Median:", calculated_median(orders))
print("Original Modes:", calculated_mode(orders))


new_orders = orders.copy()
new_orders.remove(max(new_orders))

print("\nAfter Removing Largest Order")
print("Mean:", calculated_mean(new_orders))
print("Median:", calculated_median(new_orders))
print("Modes:", calculated_mode(new_orders))
```

```
Original Mean: 9.95
Original Median: 4.0
Original Modes: [2]

After Removing Largest Order
Mean: 6.2631578947368425
Median: 4
Modes: [2]
```

```
# (b) Create two transformed datasets: add 20 to every observation in the first transformation and multiply every
# observation by 3 in the second. Calculate the mean, median, and mode for each dataset, then use Python to verify how thes
add_20 = [x + 20 for x in orders]
multiply_3 = [x * 3 for x in orders]
datasets = {
    "Original": orders,
    "Add 20": add_20,
    "Multiply by 3": multiply_3
}

print(f"{'Dataset':<16}{'Mean':<12}{'Median':<12}{'Mode'}")
print("-" * 50)

for name, data in datasets.items():
    print(
        f"{name:<16}"
        f"{calculated_mean(data):<12.2f}"
        f"{calculated_median(data):<12.2f}"
        f"{calculated_mode(data)}"
    )

print("\nVerification")
```

```
print("Mean after adding 20:",
      calculated_mean(orders) + 20)
print("Median after adding 20:",
      calculated_median(orders) + 20)

print("Mean after multiplying by 3:",
      calculated_mean(orders) * 3)
print("Median after multiplying by 3:",
      calculated_median(orders) * 3)
```

```
Dataset         Mean        Median      Mode
-------------------------------------------------
Original        9.95        4.00        [2]
Add 20          29.95       24.00       [22]
Multiply by 3   29.85       12.00       [6]

Verification
Mean after adding 20: 29.95
Median after adding 20: 24.0
Mean after multiplying by 3: 29.849999999999998
Median after multiplying by 3: 12.0
```

Q5. Missing Values and Outlier Investigation A data analyst receives the following daily sales figures, in thousands of rupees.

```
# (a) Convert the data into a Pandas Series, identify missing observations, and calculate the percentage of missing values.
#  mean, median, and all modes. Then use the IQR method to identify potential outliers.

import pandas as pd
sales = [
    12, 15, 14, 16, None,
    15, 18, 14, 17, 16,
    None, 19, 15, 120, 14
]

df= pd.Series(sales)

missing = df.isna()
missing_count = missing.sum()
missing_percentage = (missing_count / len(df)) * 100

print("Sales Series:")
print(df)

print("\nMissing observations:")
print(df[missing])
print("Missing count:", missing_count)
print("Missing percentage:", missing_percentage, "%")

# Exclude missing values
clean_sales = df.dropna().tolist()

# Reusable statistical functions
def calculated_mean(data):
    return sum(data) / len(data)

def calculated_median(data):
    data = sorted(data)
    n = len(data)

    if n % 2 != 0:
        return data[n // 2]
    return (data[n // 2 - 1] + data[n // 2]) / 2

def calculated_mode(data):
    frequency = {}

    for x in data:
        frequency[x] = frequency.get(x, 0) + 1

    highest = max(frequency.values())

    if highest == 1:
        return []

    return [x for x, count in frequency.items()
            if count == highest]


print("\nStatistics without missing values:")
print("Mean:", calculated_mean(clean_sales))
```

```
print("Median:", calculated_median(clean_sales))
print("Modes:", calculated_mode(clean_sales))

# IQR method
q1 = df.dropna().quantile(0.25)
q3 = df.dropna().quantile(0.75)
iqr = q3 - q1

lower_limit = q1 - 1.5 * iqr
upper_limit = q3 + 1.5 * iqr

outliers = [
    x for x in clean_sales
    if x < lower_limit or x > upper_limit
]

print("\nQ1:", q1)
print("Q3:", q3)
print("IQR:", iqr)
print("Lower limit:", lower_limit)
print("Upper limit:", upper_limit)
print("Potential outliers:", outliers)
```

```
Sales Series:
0      12.0
1      15.0
2      14.0
3      16.0
4       NaN
5      15.0
6      18.0
7      14.0
8      17.0
9      16.0
10      NaN
11     19.0
12     15.0
13    120.0
14     14.0
dtype: float64

Missing observations:
4    NaN
10   NaN
dtype: float64
Missing count: 2
Missing percentage: 13.333333333333334 %

Statistics without missing values:
Mean: 23.46153846153846
Median: 15.0
Modes: [15.0, 14.0]

Q1: 14.0
Q3: 17.0
IQR: 3.0
Lower limit: 9.5
Upper limit: 21.5
Potential outliers: [120.0]
```

```
# (b) Recalculate the mean, median, and mode after excluding the suspected outlier of 120. Compare the results, calculate th
# whether the outlier should be removed, retained, or investigated further.

sales_without_outlier = [
    x for x in clean_sales if x != 120
]

old_mean = calculated_mean(clean_sales)
old_median = calculated_median(clean_sales)

new_mean = calculated_mean(sales_without_outlier)
new_median = calculated_median(sales_without_outlier)
new_mode = calculated_mode(sales_without_outlier)

mean_change = abs(
    (new_mean - old_mean) / old_mean
) * 100

median_change = abs(
    (new_median - old_median) / old_median
) * 100

comparison = pd.DataFrame({
    "Measure": ["Mean", "Median"],
```

```
    "Before Exclusion": [old_mean, old_median],
    "After Exclusion": [new_mean, new_median],
    "Percentage Change": [mean_change, median_change]
})

print(comparison.round(2))
print("\nMode after exclusion:", new_mode)

print("\nRecommendation:")
print("Investigate the value 120 before removing it.")
```

```
  Measure  Before Exclusion  After Exclusion  Percentage Change
0    Mean             23.46            15.42              34.29
1  Median             15.00            15.00               0.00

Mode after exclusion: [15.0, 14.0]

Recommendation:
Investigate the value 120 before removing it.
```

Start coding or generate with AI.